

Build a Reinforcement Learning Agent for a Custom `pymunk` Physics Environment

[Back](#)**Mandatory**

Domain

Data Science

Machine Learning

Skills

Data Visualisation

Go Programming

Reinforcement Learning

Simulation

Control Systems

Environment Design

Reward Function Design

Difficulty

Hard

Tools

Docker

Matplotlib

Numpy

Pandas

Python

Pytorch

Pygame

Pymunk

Stable-Baselines3

Industries

Game Development

Robotics



Overdue - Late Submission Allowed

OVERDUE BY

4d

The deadline passed 4d ago. However, submission evaluation has not started yet. You can still submit, but it will be marked as late and may be subject to penalties.

Deadline: **9 May 2026, 04:59 pm**

[Overview](#)[Instructions](#)[Resources](#)[Submit](#)

Objective

You will build a complex, 2D physics-based control environment from scratch using `pymunk` and `pygame`. You'll learn the fundamentals of reinforcement learning (RL) environment design, including defining observation/action spaces and engineering a reward function to guide an agent's learning. By training a Proximal Policy Optimization (PPO) agent to master this environment, you will gain hands-on experience in the complete RL workflow, a critical skill set for robotics, autonomous systems, and advanced game AI.

[🔗 Report Issue](#)

Description

partnr

Welcome to the world of reinforcement learning (RL), a powerful branch of machine learning where agents learn to make decisions by interacting with an environment to maximize a cumulative reward. Unlike supervised learning, which requires labeled data, RL allows an agent to learn optimal behaviors through trial and error. This project will take you deep into the practical application of RL by challenging you to build, train, and solve a custom physics-based control problem.

The classic "CartPole" problem is the "Hello, World!" of RL, where an agent learns to balance a single pole on a moving cart. In this task, you will go significantly beyond that by designing and implementing a **Double Inverted Pendulum** environment. This system involves a cart on a track with two poles linked together, one on top of the other. The agent's goal is to apply forces to the cart to balance both poles upright simultaneously—a task that is dynamically unstable and notoriously difficult for traditional control methods.

Why is this important? Mastering such control problems is fundamental to robotics. The principles you'll apply are the same ones used to make bipedal robots walk, robotic arms manipulate objects with precision, and self-driving cars navigate complex traffic scenarios. By building the environment from scratch, you gain an intimate understanding of the physics, the agent-environment feedback loop, and the crucial art of reward function design.

2. Core Technologies and Concepts

To succeed, you'll need a solid grasp of the following tools and concepts. We'll be using a stack that is powerful yet accessible, designed to run efficiently on a standard laptop CPU.

2.1. Reinforcement Learning (RL) Fundamentals

1. **What it is:** RL is a goal-oriented learning paradigm. An **agent** interacts with an **environment** by taking **actions**. The environment responds with a new **state** (or observation) and a **reward**. The agent's objective is to learn a **policy**—a mapping from states to actions—that maximizes the total expected reward over time.
2. **Key Concepts:**
 1. **State (S):** A complete description of the environment. In our case, this will be the cart's position/velocity and the angles/angular velocities of both poles.
 2. **Action (A):** A choice the agent can make. Here, it will be the continuous force applied to the cart.
 3. **Reward (R):** A scalar feedback signal indicating how good the last action was. Designing this is a core part of your task.
 4. **Policy (π):** The agent's strategy or brain. It takes a state as input and outputs an action.

3. **Why it matters:** It's the core theoretical framework that allows us to train intelligent

partnr

2.2. pymunk : The 2D Physics Engine

1. **What it is:** `pymunk` is a high-performance, easy-to-use Python library for 2D rigid body physics simulations. It's a wrapper around the robust Chipmunk2D C library.
2. **How it works:** You create a `Space` and add `Body` objects to it. `Body` objects have physical properties like mass and inertia. You then attach `Shape` objects (like circles, polygons, or segments) to these bodies for collision detection. Finally, you connect bodies with `Constraint` objects (like `PinJoints` or `SlideJoints`) to define how they can move relative to each other.
3. **Relevance to this task:** We will use `pymunk` to simulate the entire Double Pendulum system: the cart, the track, the two poles, and the joints connecting them. `pymunk` will handle all the complex physics calculations (gravity, forces, collisions, constraints) for us, allowing us to focus on the RL aspects.

2.3. pygame : The Visualization Layer

1. **What it is:** `pygame` is a cross-platform set of Python modules designed for writing video games. It provides a simple interface for graphics, sound, and user input.
2. **Why we need it:** While `pymunk` handles the physics, it has no built-in way to visualize the simulation. We will use `pygame` to create a window and draw the state of our `pymunk` space in each frame. This visual feedback is crucial for debugging the environment and for observing the agent's behavior as it learns.

2.4. Proximal Policy Optimization (PPO): The Learning Algorithm

1. **What it is:** PPO is a state-of-the-art policy gradient RL algorithm. It's known for its stability, reliability, and excellent performance across a wide range of tasks.
2. **How it works (simplified):** PPO learns by iteratively collecting experience from the environment and using that data to update its policy. Its key innovation is the "clipped surrogate objective," which prevents the policy from changing too drastically in a single update. This clipping mechanism is what makes PPO more stable and easier to tune than many other policy gradient methods.
3. **Why we're using it:** PPO is a great choice for this task because it works well with continuous action spaces, is sample-efficient, and is robust to hyperparameter settings. We will use the implementation from the `stable-baselines3` library, a widely-used toolkit for RL research and development.

3. System Architecture and Workflow

The project consists of several interconnected components, all running within a Docker container to ensure reproducibility. The following diagram illustrates the overall architecture and data flow:

partnr

1. **Environment Setup:** The user builds the Docker image defined in the `Dockerfile`. This installs all necessary dependencies (`python` , `pymunk` , `pygame` , `stable-baselines3` , `torch` , etc.).
2. **Training Phase (`train.py`):** The user runs the training script via `docker-compose` .
 1. The script instantiates our custom `DoublePendulumEnv` and a `PPO` agent from `stable-baselines3` .
 2. The agent interacts with the environment for hundreds of thousands of timesteps, collecting experience.
 3. During training, performance metrics (like mean reward) are logged to a CSV file.
 4. At the end of training, the agent's learned policy (the neural network weights) is saved to a `.zip` file.
3. **Evaluation Phase (`evaluate.py`):** The user runs the evaluation script.
 1. It loads a pre-trained model file.
 2. It runs the agent in the environment with rendering turned on, allowing visual inspection of the learned policy.
 3. This phase can also be used to record GIFs or videos of the agent's performance.
4. **Analysis Phase:** The user takes the CSV logs generated during training and uses a script (or a Jupyter notebook) to plot the learning curves, comparing the performance of agents trained with different reward functions.

4. Step-by-Step Implementation Guide

This section provides a detailed walkthrough of the implementation process.

Part 1: Setting Up the Docker Environment

Your first step is to create a reproducible environment. This is crucial for ML projects to ensure that your code runs the same way on any machine.

1. **`Dockerfile`** : This file defines the instructions to build your Docker image. It should:
 1. Start from a base Python image (e.g., `python:3.9-slim`).
 2. Set a working directory (e.g., `/app`).
 3. Copy your `requirements.txt` file and install the dependencies. Your `requirements.txt` should include `pygame` , `pymunk` , `stable-baselines3[extra]` , `torch` , `pandas` , `matplotlib` .
 4. Copy the rest of your application code into the image.
2. **`docker-compose.yml`** : This file makes it easy to manage your Docker container. You should define services to run your main scripts.

Example docker-compose.yml structure:

partnr

```
app:
  build: .
  volumes:
    - ../app # Mount current directory for easy development

train:
  build: .
  volumes:
    - ../app
  command: python train.py --timesteps 200000

evaluate:
  build: .
  environment:
    - DISPLAY=${DISPLAY} # For GUI forwarding
  volumes:
    - /tmp/.X11-unix:/tmp/.X11-unix # For GUI forwarding
    - ../app
  command: python evaluate.py --model_path models/ppo_model.zip
```

Note on GUI forwarding: Getting `pygame` to render from inside a Docker container requires some setup to forward the X11 display. The `DISPLAY` and volume mount above are typical for Linux hosts. For Windows/macOS, you may need an X server like XQuartz or VcXsrv.

Part 2: Building the Custom `DoublePendulumEnv`

This is the heart of the project. You'll create a Python class, `DoublePendulumEnv`, that conforms to the OpenAI `gym.Env` interface (which `stable-baselines3` expects). The key methods are `__init__`, `reset`, `step`, and `render`.

1. `__init__(self, reward_type='shaped') :`
 1. Initialize `pygame` and `pymunk`.
 2. Create the `pymunk.Space` and set gravity.
 3. Define your action and observation spaces using `gym.spaces`. The observation space will be a `Box` of 6 floats (cart pos/vel, pole1 ang/vel, pole2 ang/vel). The action space will be a `Box` of 1 float (force).
 4. Store the `reward_type` to switch between reward functions.
 5. Call `self.reset()` to create the initial physical objects.
2. `reset(self) :`
 1. This method is called at the beginning of each episode. It must clear out any old physics objects and create a new simulation.
 2. Create the cart `Body` and `Shape`.
 3. Create the two pole `Body` and `Shape` objects.

4. Create the joints (`pymunk.PivotJoint`) to connect the cart to pole 1, and pole 1 to

partnr

6. Return the initial observation of the system (e.g., all values near zero).

3. `step(self, action)` :

1. This is the main agent-environment interaction loop.
2. Apply the force from the `action` to the cart body (`cart_body.apply_force_at_local_point(...)`).
3. Advance the physics simulation by one timestep (`self.space.step(dt)`).
4. Calculate the reward based on the new state of the system using the selected reward function.
5. Determine if the episode is over (e.g., if a pole falls over too far or the cart goes off-screen). This is the `done` flag.
6. Gather the new observation.
7. Return (`observation, reward, done, info_dict`).

4. `render(self, mode='human')` :

1. This method visualizes the current state.
2. Clear the `pygame` screen.
3. Iterate through the `pymunk` shapes in your space.
4. For each shape, get its position and angle, and draw it on the `pygame` screen.
5. Update the display (`pygame.display.flip()`).

Part 3: Defining Spaces and Engineering the Reward Function

1. **Observation Space:** The agent needs to know the state of the system. A good observation vector is crucial. It must contain enough information to derive a good policy. For the double pendulum, this is:

1. Cart Position (`x`)
2. Cart Velocity (`vx`)
3. Pole 1 Angle (`theta1`)
4. Pole 1 Angular Velocity (`omega1`)
5. Pole 2 Angle (`theta2`)
6. Pole 2 Angular Velocity (`omega2`)

2. **Action Space:** The agent's output. A continuous `Box(low=-1.0, high=1.0)` is a standard choice. You'll scale this value to a reasonable force magnitude in your `step` function.

3. Reward Function Design (Critical Task):

1. **Baseline Reward:** The simplest possible reward. The goal is to be upright. A good baseline is based on the vertical position of the tips of the poles. A more common approach is to use the cosine of the angles. `reward = cos(theta1) + cos(theta2)` . This gives a maximum reward of 2 when both poles are perfectly upright and less otherwise.
2. **Shaped Reward:** This is where you use your domain knowledge to help the agent learn faster. A shaped reward provides more frequent, granular feedback. You

should augment the baseline reward with penalties for undesirable behavior.

partnr

2. **Center Penalty:** `- abs(cart_x) * 0.1` (discourages running off-screen).
3. **Velocity Penalty:** `-(abs(omega1) + abs(omega2)) * 0.01` (encourages stability and smooth movements).
4. **Action Penalty:** `-(action**2) * 0.001` (discourages using excessive force, saving energy). You must experiment with the weights of these terms. Document your final function and the reasoning for each term in your `README.md`.

Part 4: Implementing the Training Script (`train.py`)

This script will orchestrate the learning process.

1. Use Python's `argparse` to accept command-line arguments for `timesteps`, `reward_type`, and `save_path`.
2. Instantiate your environment: `env = DoublePendulumEnv(reward_type=args.reward_type)`.
3. Wrap the environment with `stable_baselines3` wrappers if needed (e.g., for logging).
4. Instantiate the PPO agent: `model = PPO('MlpPolicy', env, verbose=1)`. The `'MlpPolicy'` uses a simple multi-layer perceptron, which is perfect for our vector-based observations.
5. Start the training: `model.learn(total_timesteps=args.timesteps)`.
6. Save the final model: `model.save(args.save_path)`.

Part 5: Implementing the Evaluation Script (`evaluate.py`)

This script lets you see your trained agent in action.

1. Use `argparse` to get the path to the saved model file.
2. Instantiate the environment: `env = DoublePendulumEnv()`.
3. Load the trained agent: `model = PPO.load(args.model_path, env=env)`.
4. Run an evaluation loop:

```
obs = env.reset()
for _ in range(1000):
    action, _states = model.predict(obs, deterministic=True)
    obs, reward, done, info = env.step(action)
    env.render()
    if done:
        obs = env.reset()
env.close()
```

Part 6: Analysis and Reporting

1. **Logging:** `stable-baselines3` has built-in logging callbacks that can save metrics to a monitor CSV file. Make sure this is enabled during training.

2. **Plotting:** Create a separate script or a Jupyter Notebook to generate

partnr

using pandas .

2. Use `matplotlib` to plot the `mean_reward` against `timesteps` for both runs on the same graph.
3. Include a title, axis labels, and a legend. Save the plot as a PNG file.

5. FAQ and Common Pitfalls

1. **Q: My agent isn't learning at all! The reward stays flat or goes down.**

1. **A:** This is the most common issue in RL. Check the following: 1) **Reward Function:** Is your reward signal meaningful? An agent will exploit any loophole. Maybe your penalties are too high, or your positive reward is too sparse. Visualize the reward values being returned. 2) **Observation Space:** Does the agent have all the information it needs? Forgetting to include velocities is a common mistake. Normalize your observation values if they have very different scales. 3) **Hyperparameters:** PPO is robust, but sometimes you need to tune the learning rate or batch size. Check the `stable-baselines3` documentation for guidance. [Stable Baselines3 RL Tips](#)

2. **Q: The `pymunk` simulation is exploding or behaving weirdly.**

1. **A:** Physics simulations can be unstable if not set up correctly. 1) **Timestep:** Are you using a fixed and reasonably small timestep (`dt`) for `space.step(dt)` ? 1/60th of a second is standard. 2) **Mass/Inertia:** Do your objects have realistic mass and moment of inertia values? If masses are too different or inertia is too low, you can get unstable rotations. 3) **Joints:** Are your joints configured correctly? Check the anchor points. A misplaced anchor can cause bizarre behavior. Refer to the [Pymunk documentation](#).

3. **Q: What is the difference between the `baseline` and `shaped` reward? Why do we need both?**

1. **A:** The `baseline` reward defines the fundamental goal (e.g., stay upright). It's often sparse, meaning the agent only gets a positive reward in very specific states. A `shaped` reward adds intermediate rewards/penalties to guide the agent. For example, it penalizes wiggling too much, even if the agent doesn't fall. This can significantly speed up learning. This task requires you to implement both to demonstrate you understand this crucial concept of **reward shaping**. See this classic paper on the topic: [Policy search for motor primitives in robotics](#)

4. **Q: How do I save a GIF of the evaluation?**

1. **A:** In your `evaluate.py` script, after each call to `env.render()` , you can grab the `pygame` screen surface and save it as an image file (e.g., using `pygame.image.save`). Collect these images frame by frame and then use a library

like `imageio` to compile them into a GIF. `import imageio;`

partnr

Core Requirements

1. The project must be fully containerized using Docker for reproducible setup and execution.

Contract Specification:

1. **File Existence:** The root of the repository must contain:

1. `Dockerfile`
2. `docker-compose.yml`

2. **Functionality:**

1. The command `docker-compose build` must successfully build the application image without errors.
2. A service must be defined in `docker-compose.yml` that can run the training script (e.g., `docker-compose run train`).
3. A service must be defined that can run the evaluation script (e.g., `docker-compose run evaluate`).

Verification:

1. Check for the existence of `Dockerfile` and `docker-compose.yml`.
2. Run `docker-compose build` and verify it completes successfully.
3. Run the training service and verify that the training process starts.
4. Run the evaluation service (with a dummy model if needed) and verify it attempts to launch the simulation.

-
2. A custom reinforcement learning environment for a double inverted pendulum must be implemented using `pymunk`.

Contract Specification:

1. **File Structure:** There must be a Python module (e.g., `environment.py`) containing a class named `DoublePendulumEnv`.
2. **Interface:** The `DoublePendulumEnv` class must implement the following methods, similar to the OpenAI Gym/Gymnasium API:
 1. `__init__(self, ...)`
 2. `reset(self)` : Returns an initial observation.
 3. `step(self, action)` : Returns (observation, reward, done, info).
 4. `render(self, mode='human')`
3. **Physics Simulation:** The `step` method must use `self.space.step(dt)` to advance a `pymunk.Space` instance. The space must contain `pymunk` bodies, shapes, and joints

representing a cart, a track, and two interconnected poles.

partnr

1. Check for the existence of the environment file and the `DoublePendulumEnv` class.
2. Programmatically import the class and instantiate it.
3. Verify the existence of `reset`, `step`, and `render` methods.
4. Inspect the instance to confirm it contains a `pymunk.Space` object.

3. The environment must correctly define the observation and action spaces.

Contract Specification:

1. **Observation Space:** The `DoublePendulumEnv` instance must have an `observation_space` attribute. This must be a `gym.spaces.Box` object with `shape == (6,)`.
2. **Action Space:** The `DoublePendulumEnv` instance must have an `action_space` attribute. This must be a `gym.spaces.Box` object with `shape == (1,)` and bounds between -1.0 and 1.0 (inclusive).

Verification:

1. Instantiate `DoublePendulumEnv`.
2. Check `env.observation_space.shape` to be `(6,)`.
3. Check `env.action_space.shape` to be `(1,)`.
4. Check that `env.action_space.low` is `-1.0` and `env.action_space.high` is `1.0`.

4. The environment must implement a 'baseline' reward function.

Contract Specification:

1. **Initialization:** The `DoublePendulumEnv` class's `__init__` method must accept an argument like `reward_type`.
2. **Functionality:** When initialized with `reward_type='baseline'`, the reward returned by the `step` method must primarily be a function of the poles' angles (e.g., based on the cosine of the angles), rewarding upright positions.

Verification:

1. Instantiate the environment: `env = DoublePendulumEnv(reward_type='baseline')`.
2. Call `env.reset()` and then `env.step(action)` with the system in a known state (e.g., upright).
3. Verify that the returned reward is positive and maximal in the upright state.

- 5.

The environment must implement a 'shaped' reward function that provides more

partnr

1. **Initialization:** The environment must be initializable with `reward_type='shaped'`.
2. **Functionality:** When using the 'shaped' reward, the reward calculation in the `step` method must include at least two additional terms beyond the baseline upright bonus. Examples include penalties for cart distance from center, high angular velocities, or high action magnitude.
3. **Documentation:** The `README.md` file must contain a section explaining the components of this shaped reward function.

Verification:

1. Instantiate `env = DoublePendulumEnv(reward_type='shaped')`.
2. Programmatically verify that the reward signal changes when cart position or angular velocities change, even if the pole angles are constant.
3. Check the `README.md` for the required documentation section.

-
6. A script must be provided to train the PPO agent.

Contract Specification:

1. **File Existence:** A `train.py` script must exist in the repository.
2. **Functionality:** The script must:
 1. Use a library like `stable-baselines3` to create a PPO agent.
 2. Train the agent on the `DoublePendulumEnv`.
 3. Be configurable via command-line arguments (e.g., `python train.py --reward_type shaped --timesteps 1000 --save_path models/test.zip`).
 4. Save the trained model to a specified file path upon completion.

Verification:

1. Check for `train.py`.
2. Run the script with a small number of timesteps (e.g., `--timesteps 10`).
3. Verify that the script completes without errors and creates a model file at the specified path.

-
7. A script must be provided to load a trained model and visualize its performance.

Contract Specification:

1. **File Existence:** An `evaluate.py` script must exist in the repository.
2. **Functionality:** The script must:

partnr

1. Accept a command-line argument for the path to a saved model file (e.g.,

3. Run the model in the `DoublePendulumEnv` .

4. Open a `pygame` window and render the simulation, showing the agent controlling the pendulum.

Verification:

1. Check for `evaluate.py` .
2. Train a dummy model for a few steps and save it.
3. Run `evaluate.py` with the path to the dummy model.
4. Verify that a `pygame` window appears. This may require a virtual framebuffer like `xvfb` in an automated environment.

8. The training script must log key metrics to a file.

Contract Specification:

1. **Output:** The `train.py` script must generate a log file, preferably in CSV format, in a `logs/` directory.
2. **Content:** The log file must contain columns for `timesteps` and `mean_reward` , tracking the agent's performance over the course of training.

Verification:

1. Run `train.py` for a small number of timesteps.
2. Check for the creation of a `logs/` directory and a CSV file within it.
3. Parse the CSV file and verify that it contains the required columns (`timesteps` , `ep_rew_mean` or similar).

9. A plot comparing the learning curves of the two reward functions must be present in the repository.

Contract Specification:

1. **File Existence:** The repository must contain a file named `reward_comparison.png` in the root directory.
2. **Content:** The image must be a 2D plot showing 'Mean Reward' on the y-axis and 'Timesteps' on the x-axis. It must display two distinct lines: one for the training run with the `baseline` reward and one for the `shaped` reward. The plot must include a title, axis labels, and a legend.

Verification:

1. Check for the existence of `reward_comparison.png`.

partnr

10. The repository must include GIFs demonstrating the agent's performance at different stages of training.

Contract Specification:

1. **File Existence:** A `media/` directory must exist.
2. **Content:** The `media/` directory must contain at least two GIF files:
 1. `agent_initial.gif`: Showing the performance of an agent early in training (e.g., after < 50,000 steps).
 2. `agent_final.gif`: Showing the performance of the fully trained agent.

Verification:

1. Check for the existence of `media/agent_initial.gif` and `media/agent_final.gif`.
-

11. A comprehensive README.md file must document the project.

Contract Specification:

1. **File Existence:** A `README.md` file must be present in the root of the repository.
2. **Content:** The file must contain the following markdown sections (headers):
 1. `### Environment Design`: Describing the `pymunk` setup.
 2. `### Reward Function Design`: Explaining the mathematical formulation and rationale for both the `baseline` and `shaped` reward functions.
 3. `### How to Run`: Providing clear, step-by-step instructions for building the Docker image and running the training and evaluation scripts.

Verification:

1. Check for the existence of `README.md`.
 2. Parse the markdown file to ensure the required sections are present.
-

12. An `.env.example` file must be provided to document all environment variables.

Contract Specification:

1. **File Location:** `.env.example` must be in the repository root.
2. **Content:** The file must document any and all environment variables required to configure and run the application (e.g., training parameters, file paths). If no environment variables are used, the file can be empty but must exist.

Verification:

p a r t n r

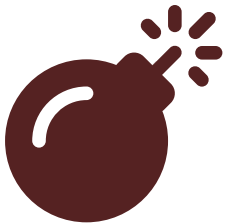
[About Us](#)

[Contact Us](#)

[Privacy Policy](#)

[Terms and Conditions](#)

All rights reserved. Copyright, Partnr 2025-26



Syntax error in text
mermaid version 11.12.3



Syntax error in text
mermaid version 11.12.3